Bridge Day 13 INSTRUCTOR KEY

Loop Design, Break, Continue, and Nested Tracing

Learning sequence: Predict - Trace - Explain - Test - Interpret

Wednesday, July 22, 2026 • 50 points • longitudinal template 07242026_v1

Name		**UIN**		**Section**	

Daily target and independence boundary

Select skip, stop, and reset behavior while tracing repeated work inside repeated work.

Allowed tools	Prior tools plus break, continue, and nested for-loop tracing
Support supplied	Behavioral requirements and a sample input stream are supplied.
Student decides	Loop form, placement of skip and stop decisions, state updates, and reset scope.
Scaffold level	Requirements only; students select the control-flow pattern.

Evidence and scoring

Evidence	**Points**	**Secure work shows**
Integrated trace	10	intermediate state, condition results, exact output, and meaning
Diagnosis and repair	8	actual, intended, cause, repair, and a discriminating test
Complete program and tests	32	coherent executable logic, required output, assumptions, and defended tests

Task 1. Integrated trace - 10 points

Trace nested repetition while distinguishing a skipped pass, an early stop, and a per-station reset.

```
overall = 0

for station in range(1, 3):
    station_total = 0
    for reading in [0, station + 1, 5]:
        if reading == 0:
            continue
        if reading == 5:
            break
        station_total = station_total + reading
    overall = overall + station_total
    print(station, station_total)

print(overall)
```

Your task

1. Give a complete reached-pass ledger that identifies the continue, break, station_total reset, and overall update.
2. Write every output line and explain why the inner break does not end the outer loop.

Answer 1. Station 1 resets station_total to 0. Reading 0 continues; reading 2 is added; reading 5 breaks the inner loop. The station total is 2 and overall becomes 2. Station 2 resets station_total to 0. Reading 0 continues; reading 3 is added; reading 5 breaks the inner loop. The station total is 3 and overall becomes 5.
Answer 2. Exact output: 1 2; 2 3; 5. break ends only the innermost loop that contains it, so the outer station loop advances from station 1 to station 2.

Task 2. Diagnose and repair - 8 points

The intent is to print every odd integer through 5 and then stop. The current order omits 5.

```
value = 0
while value < 8:
    value = value + 1
    if value % 2 == 0:
        continue
    if value == 5:
        break
    print(value)
```

Your task

1. Give actual versus intended output, identify the ordering cause, and write a minimal repair that still stops at 5.

Answer 1. Actual output is 1 and 3. Intended output is 1, 3, and 5. The break occurs before print when value is 5. Minimal repair: move print(value) before the if value == 5 test, then keep break after the print. Even values still continue before printing.

Task 3. Construct a complete program - 32 points

Program specification

- Repeatedly read an integer inspection code.
- Code 0 ends normal input. A negative code is ignored and processing continues.
- A code of 90 or greater is a shutdown code: stop immediately without counting or adding it.
- Each positive code below 90 is accepted. Count accepted codes and add their values.
- After the loop, print accepted_count and total on separate lines.

Program workspace

Model program

```
accepted_count = 0
total = 0

while True:
    code = int(input())
    if code == 0:
        break
    if code < 0:
        continue
    if code >= 90:
        break
    accepted_count = accepted_count + 1
    total = total + code

print(accepted_count)
print(total)
```

Reasoning: The loop reads once at the top of every pass. Because no input update is placed after continue, the next pass naturally returns to input instead of repeating stale state.

Reasoning: The normal sentinel and shutdown condition both use break but have different meanings. Neither contributes to the count or total.

Task 3 continued. Test and defend - included in 32 points

Continue code if needed.

Expected tests and results

1. Inputs 12, -3, 25, 99, 40 produce accepted_count 2 and total 37; 40 is never read after shutdown.
2. Inputs -5, 8, 0 produce 1 and 8. Immediate 0 produces 0 and 0.

Required tests, expected results, and brief defense

Scoring guidance: 6 input/loop architecture; 4 normal sentinel; 4 continue case; 4 shutdown break; 6 state updates; 3 output; 5 tests and control-flow explanation.

Bridge Day 13 INSTRUCTOR KEY

VIP Evidence Handoff

Learning sequence: Predict - Trace - Explain - Test - Interpret

Contract 07a04826c975 • longitudinal template 07242026_v1

Why engineers use this page

Select a loop form from the requirement, complete a sample/transfer pair, then finish the visibly labeled Independent Program Studio whose final build is a complete input-to-output loop program.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: for loop, while loop, range call, loop state update, termination condition, list literal as collector, append call

Carried authoring tools: assignment, print call, arithmetic expression, modulo, input call, int conversion, f string, comparison expression, if elif else

Supplied-code exposure only: list index read

An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.18	Multiples In Range	`18-18-work / 18-18-transfer / 18-18-cold`	append call, arithmetic expression, for loop, range call
18.19	Countdown To Multiple	`18-19-work / 18-19-transfer / 18-19-cold`	arithmetic expression, while loop
18.20	Smallest Value	`18-20-work / 18-20-transfer / 18-20-cold`	comparison expression, for loop, if elif else
18.21	Cooldown Sequence	`18-21-work / 18-21-transfer / 18-21-cold`	append call, arithmetic expression, while loop
18.22	Rounds To Clear Queue	`18-22-work / 18-22-transfer / 18-22-cold`	arithmetic expression, f string, input call, int conversion, while loop

Readiness evidence

1. Classify each activity as fixed traversal or condition-controlled repetition and justify one classification.

   Model response: 18.18 and 18.20 use fixed traversal; 18.19, 18.21, and 18.22 are condition-controlled. The justification must point to a known collection/range or an unknown pass count.

2. Choose one activity and list the complete state that must still be correct after the loop ends.

   Model response: Accept activity-specific final state, including the collected list, first matching number, smallest value, full cooldown sequence, or rounds/backlog state.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	A specific expected state or output before execution.
Observed Result	The actual state or output, including a value or exact text.
Revision Reason	The defect or mismatch and the change that addresses it.
Engineering Meaning	How the evidence changes or supports the engineering decision.
Units Or Not Applicable	The physical unit, or the evidence type when no unit applies.
Assumption	One condition accepted as true for this model or rule.
Model Limit	One situation the result does not justify or predict.
Next Test	A concrete boundary, invalid, or alternate case with an expected result.
Help Trigger	The exact unresolved point that would trigger a targeted question.
Minutes Used	Approximate minutes from first prediction through revision.

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Generated from the canonical VIP construct and evidence contract; workbook planning and executable notebook work remain distinct.